

Documentación del sistema de lealtad de XRE Docs

Entregable de Transformación Digital y Seguridad y Auditoría de Sistemas

Integrantes: Victor Estuardo Florian, Dino Fiagioli, Jorge Otoniel Castillo Ortega.

Fecha de la evaluación: 8 de octubre de 2026. Las marcas Nine West y Rimet corresponden a catálogos del sistema. Las tiendas y cuentas con la etiqueta Demo XRE son datos de demostración del proyecto.

El sistema automatiza el registro de compras, el cálculo de puntos y el canje de premios. Una compra de Q500 generó 500 puntos, un premio consumió 200 y el cliente conservó 300 puntos disponibles. El historial identifica a quienes aprobaron y entregaron el premio.

La evaluación produjo 37 verificaciones de API aprobadas, 23 pruebas automatizadas aprobadas y una restauración de PostgreSQL con conteos coincidentes en 53 tablas. Estas evidencias prueban el escenario ejecutado en Docker local.

El paquete contiene este informe, manual con capturas, presentación de 8 minutos, guion, evidencias, diagramas UML editables y código fuente organizado. El despliegue público con HTTPS y los avisos de dependencias permanecen pendientes de validación y remediación.

Contenido: proceso y solución, arquitectura y UML, código fuente, despliegue, seguridad, pruebas, resultados, conclusiones y matriz de cumplimiento. Cada apartado enlaza la explicación con archivos y resultados concretos.

Proceso seleccionado y problema

Fidelización mediante compras y canjes de premios

XRE Docs presenta una aplicación para administrar un programa de lealtad en comercios con varias tiendas. El proceso seleccionado inicia al identificar al cliente y registrar su factura. Continúa con la acreditación de puntos y termina con la entrega o cancelación del premio solicitado.

Como escenario de referencia para justificar la automatización, un control manual necesita consultar facturas, calcular puntos y coordinar entregas en registros separados. Esa forma de trabajo puede producir duplicaciones, diferencias de saldo y dificultad para determinar quién autorizó un beneficio. El proyecto no incluye un estudio histórico de tiempos o pérdidas de la empresa.

La necesidad funcional consiste en mantener un saldo verificable por cliente, aplicar una regla común y limitar las operaciones por permiso y tienda. El cliente debe consultar sus beneficios sin depender de una consulta manual al personal.

La automatización centraliza las validaciones en la API. La tienda activa procede de la sesión, las facturas tienen una restricción de unicidad por tienda y los puntos derivan de movimientos registrados. El cliente puede seguir el estado de sus solicitudes.

Objetivo del proyecto: completar el ciclo de fidelización con trazabilidad y controles verificables. Criterios de aceptación: compra válida, saldo correcto, canje con stock, acciones autorizadas y evidencia de auditoría.

Etapa	Riesgo del control manual	Respuesta de la aplicación
Identificar cliente	Confundir cuentas o mantener duplicados	Código generado y validación de correo, teléfono y NIT
Registrar factura	Duplicación o cálculo inconsistente	Número único por tienda y regla en servidor
Solicitar premio	Canjear sin saldo o sin disponibilidad	Validación de puntos y reserva de stock
Entregar premio	Falta de responsable o doble entrega	Flujo de estados, permiso y registro del receptor

Solución desarrollada y funcionalidades

Una API central coordina los portales y el programa de puntos

La web administrativa permite gestionar clientes, usuarios, permisos, tiendas, catálogos, reglas, productos canjeables y auditoría. El personal de tienda registra facturas y atiende canjes según sus accesos directos y su tienda asignada.

El portal cliente permite autenticarse, consultar puntos y nivel, explorar premios y revisar compras y canjes propios. También incorpora perfiles, notificaciones y funciones de tienda online. La etiqueta PWA identifica el proyecto de interfaz móvil; esta evaluación no acredita funcionamiento sin conexión.

La ampliación de comercio online incluye marcas, productos, carrito, pedidos, agenda de entregas, mensajeros y liquidación de cobros. Existe un portal de repartidores. Las pruebas automatizadas cubrieron reglas de pago, entrega y liquidación. No se ejecutó en esta evaluación un pedido online completo desde la interfaz.

Flujo demostrado: registro del cliente, selección de tienda, previsualización y registro de Q500, acreditación de 500 puntos, solicitud de premio de 200 puntos, aprobación, envío a tienda, preparación y entrega. La cancelación de una segunda solicitud devolvió sus puntos. La reversa de otra factura conservó el historial.

La regla vigente utiliza Q1 por punto, redondeo hacia abajo y máximo de 1000 puntos por compra. Su valor de referencia es Q0.01 por punto. Las cifras de la demostración dependen de esta configuración y no representan ventas reales.

Arquitectura y límites de confianza

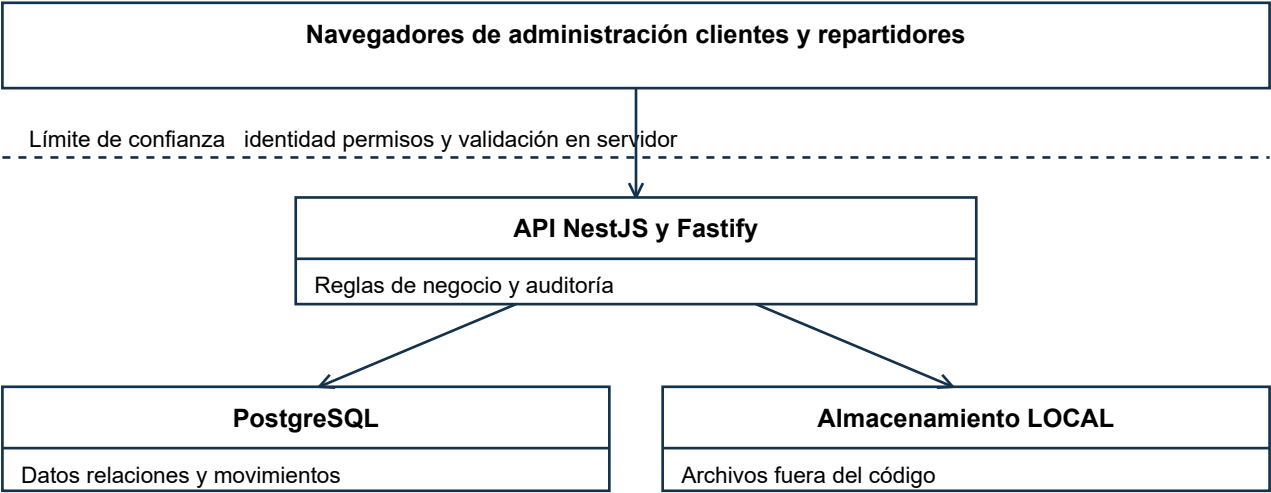
Separación entre navegador, servicios y almacenamiento

El monorepositorio contiene interfaces Next.js con React y TypeScript, una API NestJS sobre Fastify y PostgreSQL 16 gestionado mediante Prisma. La API ejecuta reglas de negocio, autenticación, autorización, acceso a datos y auditoría.

Las interfaces envían solicitudes mediante /proxy/api. Los archivos next.config.mjs reescriben esa ruta hacia la API interna. En el entorno local, los puertos de administración, clientes, repartidores, API y PostgreSQL están publicados por Docker.

El navegador constituye una entrada no confiable. La API debe verificar identidad, permisos, pertenencia a tiendas, formatos y reglas comerciales. La base de datos guarda relaciones y restricciones. El almacenamiento LOCAL conserva archivos en un volumen separado del código.

La configuración de producción añade Caddy para TLS y MinIO para objetos. Ese diseño pretende publicar únicamente el proxy de entrada. Caddy, MinIO y el servicio de respaldos no forman parte del entorno local ejecutado.



Componente	Tecnología o responsabilidad
Web y portales	Next.js 16 y React 18
API	NestJS 11, Fastify, Zod, Argon2 y JWT
Persistencia	PostgreSQL 16 y Prisma 5
Infraestructura	Docker Compose y volúmenes persistentes
Producción prevista	Caddy y almacenamiento S3 compatible con MinIO

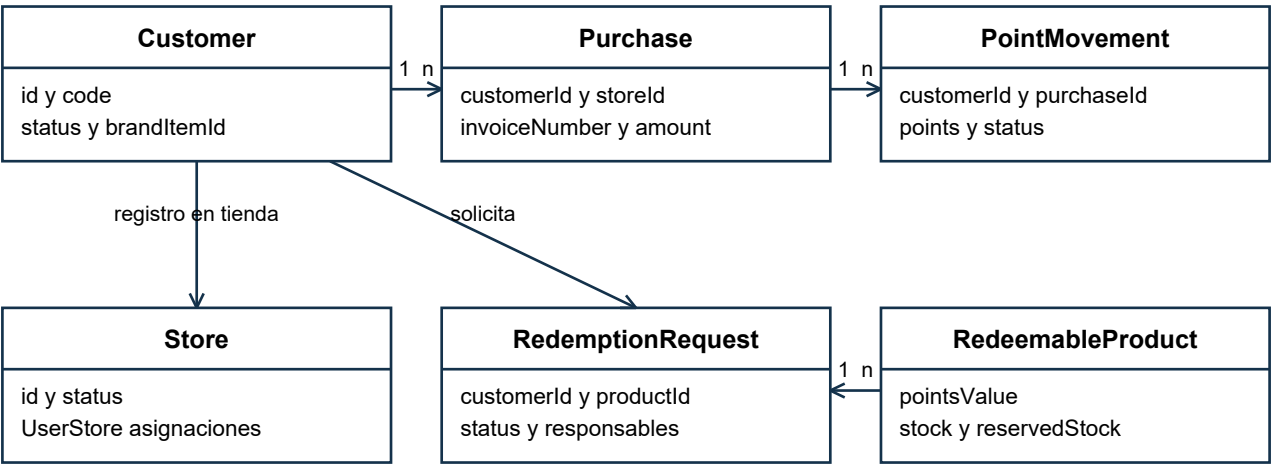
Modelo UML y permisos

Las relaciones sostienen la trazabilidad del programa

El modelo relaciona cada compra con un cliente, una tienda y un usuario interno. La compra genera movimientos de puntos. El canje relaciona al cliente con un producto canjeable y una tienda de retiro. Su historial conserva responsables de aprobación, preparación y entrega.

Los usuarios internos tienen asignaciones de roles, tiendas y permisos directos. En el guard vigente, los permisos efectivos se toman de InternalUserPermission. El nombre del rol por sí solo no concede los accesos; se deben comprobar los permisos directos del usuario.

Los diagramas editables se entregan en diagramas/modelo-clases.mmd, diagramas/casos-uso.mmd y diagramas/secuencia-canje.mmd. Las figuras simplifican atributos para mostrar las relaciones críticas.



Actor	Acciones demostradas	Restricción verificada
Cliente	Saldo propio, solicitud y cancelación	No accede a rutas internas
Operador de tienda	Factura, preparación y entrega	7 permisos y una tienda asignada
Revisor	Aprobación, reversa y auditoría	20 permisos explícitos para la demo
Administrador del programa	Usuarios, reglas, catálogos y reportes	Asignación explícita de permisos

Secuencia crítica del canje

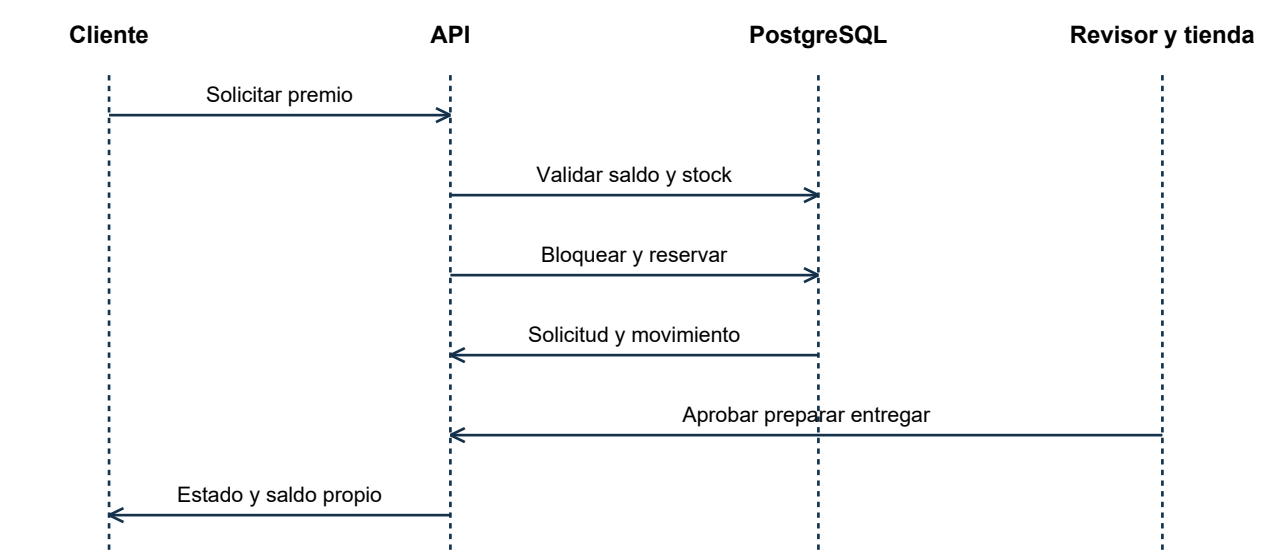
La API protege el saldo y la disponibilidad durante la solicitud

El cliente envía el producto y la tienda de retiro con su sesión. La API obtiene su identidad de la sesión, comprueba que el cliente y el premio estén activos y verifica publicación, marca, stock y saldo.

Dentro de una transacción, el servicio toma bloqueos de PostgreSQL por cliente y por producto. Vuelve a leer saldo y stock, registra la solicitud y crea el movimiento negativo de puntos. Para premios que requieren aprobación, reserva disponibilidad mediante reservedStock.

El revisor aprueba y envía a tienda. El operador prepara y entrega con su permiso y su tienda activa. El servidor rechaza una segunda entrega. La cancelación recupera puntos y libera la reserva correspondiente.

La revisión de código acredita los bloqueos de concurrencia. La evaluación funcional demuestra una solicitud individual, una cancelación y una entrega repetida rechazada. No equivale a una prueba de carga ni a una prueba concurrente del último producto.



Comprobación	Evidencia concreta
Saldo insuficiente	T21 devolvió HTTP 400
Operador sin aprobación	T23 devolvió HTTP 403
Solicitud y estados operativos	T22 y T24 a T27 devolvieron HTTP 201
Entrega repetida	T28 devolvió HTTP 400
Cancelación	T29 devolvió HTTP 201 y saldo neto final 300

Código fuente organizado

El archivo codigo-fuente.zip permite revisar la implementación

El código entregado corresponde al árbol de trabajo de la aplicación, incluidas sus modificaciones locales vigentes. El inventario codigo-fuente-manifiesto.json registra tamaño y SHA256 de cada archivo. Los secretos operativos, credenciales de demostración, datos de base y dependencias instaladas se excluyen.

La instalación reproducible utiliza package.json y package-lock.json. Desde la carpeta del código: npm ci, npm run db:generate y npm run build. Docker permite ejecutar la aplicación sin reutilizar node_modules de otra plataforma. Las dependencias copiadas de macOS dieron un error de esbuild en Windows durante el primer intento de pruebas.

Las 23 pruebas existentes se ejecutaron con Node y un cargador TypeScript que evita ese binario incompatible. El archivo reproducibilidad/ts-register.cjs documenta el cargador. Las verificaciones de tipos de API y portal cliente finalizaron sin errores.

La corrección aplicada al portal cliente mantiene Canjes y Perfil después de actualizar los datos cuando el catálogo online está vacío. Su compilación de producción y navegación desde la interfaz se verificaron. El resto del código previo se conserva en el paquete.

Ruta	Componente principal
apps/api/src/modules/auth	Identidad, cookies, sesiones, guard de permisos
apps/api/src/modules/purchases	Registro, previsualización, aprobación y reversa
apps/api/src/modules/redemptions	Reserva, estados, cancelación y entrega
apps/api/src/modules/rewards	Productos canjeables y segmentación
apps/api/src/modules/client	Datos del cliente autenticado
apps/api/src/modules/audit	Consulta y registro de acciones
apps/api/src/modules/store-catalog	Comercio online, pagos, entregas y liquidación
apps/admin-web	Interfaz administrativa y operación de tienda
apps/client-pwa y apps/driver-pwa	Portales de cliente y repartidor
packages/database/prisma	Modelo, migraciones y cargas iniciales
docker y docker-compose*.yaml	Construcción, proxy y operación
apps/api/test	Pruebas automatizadas de reglas críticas

Entorno ejecutado y operación local

La evidencia de despliegue corresponde a Docker en Windows

Se comprobaron cinco servicios activos: administración, API, portal cliente, portal repartidor y PostgreSQL. La API y PostgreSQL reportaron estado healthy. /api/health comprobó base de datos y almacenamiento disponibles.

Aunque APP_ENV tiene el valor production para la ejecución del artefacto compilado, el acceso observado utiliza HTTP local y COOKIE_SECURE=false. La modalidad de ejecución no acredita un dominio público, certificados TLS ni exposición de producción validada.

Inicio en Windows: docker compose -f docker-compose.yml -f docker-compose.local.yml up -d api admin-web client-pwa driver-pwa. Los archivos Iniciar aplicación.cmd e iniciar-local.ps1 ofrecen la misma operación.

Después de cambios: docker compose -f docker-compose.yml -f docker-compose.local.yml up -d --build api admin-web client-pwa driver-pwa. Estado: el mismo comando con ps. Diagnóstico: logs --tail=100. Las migraciones se aplican mediante prisma migrate deploy al iniciar el contenedor API.

Los volúmenes persistentes conservan PostgreSQL y el almacenamiento LOCAL. Detener con detener-local.ps1 conserva los datos. No utilizar down -v sobre información que se deba mantener. La carga de geografía reemplaza los catálogos correspondientes y se debe reservar para una base preparada para esa operación.

Servicio	Dirección local	Comprobación
Administración	http://localhost:3000	Sesión, panel, clientes y canjes
Clientes	http://localhost:3002	Saldo y catálogo propios
Repartidores	http://localhost:3003	Servicio iniciado
Salud de API	http://localhost:4000/api/health	Base y storage disponibles
PostgreSQL	Puerto local 5432	Servicio healthy y respaldo restaurado

Procedimiento para producción pública

Configuración disponible y evidencias aún requeridas

El repositorio incluye `docker-compose.production.yml`, `docker/Caddyfile` y `docs/production.md`. El archivo de producción elimina la publicación directa de puertos de PostgreSQL, MinIO, API, administración y portal cliente, y publica Caddy en 80 y 443.

Preparación: disponer de un servidor Docker, configurar dominios y DNS, crear `.env.production` desde su ejemplo y cargar secretos nuevos mediante un canal privado. El archivo nunca debe formar parte del repositorio ni de la entrega académica.

Ejecución prevista: `npm run production:config`, `npm run production:up` y `npm run production:status`. Caddy necesita dominios que resuelvan al servidor para obtener certificados. `COOKIE_SECURE` debe estar habilitado junto con HTTPS real.

Validación posterior: comprobar HTTPS, proxy `/proxy/api`, cookies, origen permitido, sesiones, aislamiento de servicios, restauración y acceso de los roles. La configuración prevista sólo define dominios de administración, cliente y API. El portal de repartidores necesita su ruta HTTPS y configuración de origen antes de incorporarlo al despliegue público.

El entregable no contiene una URL pública validada. La demostración en clase puede usar la implementación local y sus capturas. Para acreditar puesta en producción pública se deben añadir las evidencias de la tabla una vez ejecutadas.

Tarea pendiente	Evidencia de cierre	Responsable propuesto
Actualización de dependencias	Auditoría sin avisos críticos o altos aceptados	Equipo de desarrollo
Servidor y DNS	Dominio resolviendo y servicios activos	Operación
TLS y cookies Secure	Certificado válido y atributos verificados	Operación y seguridad
Aislamiento y principal DB	Puertos privados y usuario sin superusuario	Operación
Portal repartidor	Ruta HTTPS y pruebas de login y entregas	Desarrollo
Backups completos	DB y objetos restaurados desde copia externa	Operación
Pruebas de aceptación	Flujo y controles sobre el despliegue público	Equipo del proyecto

Seguridad de identidad y aplicación

Controles comprobados en código y en respuestas de la API

Autenticación: Argon2 protege las contraseñas y los hashes de refresh. Los guards verifican JWT, tipo de identidad, sesión vigente, revocación y usuario activo. Se observaron cookies de acceso y renovación con HttpOnly y SameSite=Lax. En HTTP local no llevan Secure.

Autorización: PermissionsGuard exige los permisos declarados por cada ruta. El operador de demostración no pudo consultar auditoría ni aprobar un canje. La tienda no asignada produjo HTTP 403. La identidad del cliente procede de su propia sesión.

Sesiones: la renovación de refresh pasó, la reutilización de un refresh anterior produjo HTTP 401 y revocó la sesión. El cierre de sesión invalidó el acceso usando la cookie previa. El límite de login devolvió HTTP 429 durante intentos repetidos.

CSRF: en operaciones protegidas que utilizan cookies, el guard exige coincidencia entre la cookie CSRF y x-csrf-token. La solicitud sin esa cabecera produjo HTTP 403. El token CSRF debe ser legible por la interfaz; las cookies que contienen tokens de acceso y refresh son HttpOnly.

Validación y errores: Zod rechaza campos no previstos, identificadores y montos inválidos. Los guard y servicios devuelven errores funcionales definidos. Fastify aplica Helmet y un límite de cuerpo de 16 MiB. La configuración CORS usa una lista de orígenes. Esta revisión no acredita cobertura total de endpoints ni resistencia ante todas las clases de ataque.

Seguridad de datos y continuidad

Integridad transaccional y restauración comprobada

La base establece unicidad para correo, teléfono, código de cliente y factura por tienda. El registro de compra y su movimiento de puntos ocurren dentro de una transacción. La reversa crea una compensación y conserva los hechos previos. El saldo visible deriva de los movimientos disponibles.

Los canjes toman bloqueos por cliente y producto para reducir doble gasto y sobre-reserva. La prueba de cancelación devolvió puntos y disponibilidad. La prueba de entrega repetida falló. Estos controles se deben complementar con pruebas de concurrencia y carga antes de una operación pública.

Privacidad: las rutas de cliente utilizan CustomerJwtAuthGuard y no toman customerId de la petición para consultar el resumen. T18 verificó que enviar el ID de otro cliente no cambió la identidad ni el saldo de la sesión alterna. Las capturas de clientes se filtraron para mostrar únicamente cuentas Demo XRE.

Respaldo: se ejecutó pg_dump en formato custom, se calculó SHA256 y se restauró mediante pg_restore en una base temporal independiente. Los conteos coinciden en las 53 tablas. La base temporal se eliminó al finalizar. El dump permanece privado y no forma parte del entregable.

Alcance de continuidad: la prueba anterior cubrió PostgreSQL. El código incluye backup y restore-test de base y MinIO, pero MinIO no estaba activo. Falta acreditar restauración de archivos LOCAL, cifrado de copias, almacenamiento externo, retención operativa y objetivos de recuperación.

Principal de datos: la inspección real mostró que el usuario de conexión lealtad tiene superusuario, creación de bases y creación de roles. Debe separarse un usuario de ejecución de privilegios limitados y otro de migraciones. La existencia de permisos de aplicación no corrige ese exceso de privilegios en PostgreSQL.

Archivos: la API limita tamaño y admite tipos declarados de imagen y comprobantes. La revisión no encontró una verificación de contenido por firma binaria en storeFile. Los comprobantes requieren además revisar la autorización de descarga; getAssetContent devuelve contenido por identificador.

Seguridad de DevOps y dependencias

Evidencia actual y plan de mejora antes de publicar

El archivo `.gitignore` excluye `.env`, `.env.*`, `node_modules`, artefactos de construcción, logs y respaldos. Los ejemplos de variables sirven como plantillas y no se deben usar como credenciales de operación. La protección de accesos al repositorio y de ramas depende del servidor Git y no se pudo acreditar con la copia local.

La construcción usa npm ci con `package-lock.json` y `Dockerfiles` por servicio. Las compilaciones de API y portal cliente se completaron al reconstruir la corrección de navegación. El árbol revisado no contiene un workflow de CI/CD activo. La carpeta reproducibilidad incluye una propuesta de pipeline claramente identificada.

La auditoría npm audit --omit=dev reportó 11 avisos en el conjunto de dependencias de producción: 1 crítico, 9 altos y 1 moderado. Next.js aparece con severidad crítica. El informe JSON conserva los paquetes, avisos y correcciones disponibles. Esta clasificación es la del registro consultado y no demuestra explotación de la aplicación.

Antes de publicar, actualizar dependencias compatibles, revisar los avisos transitivos y ejecutar nuevamente compilación, pruebas de tipos, pruebas unitarias y el flujo de aceptación. Evitar publicar mientras permanezcan avisos críticos o altos sin evaluación y tratamiento explícitos.

CI propuesto: checkout con permisos de lectura, instalación mediante npm ci, generación de Prisma, pruebas de tipos, pruebas funcionales automatizadas y auditoría de dependencias. La etapa de despliegue requiere secretos del entorno y validación en staging. La propuesta no acredita una ejecución de CI.

La evidencia de configuración contiene únicamente valores no confidenciales. Los resultados omiten valores de cookies y claves. El paquete de entrega excluye credenciales demo, archivos `.env` operativos y el dump de respaldo.

Paquete o grupo	Severidad observada	Tratamiento requerido
next	Crítica	Actualizar y validar navegación y proxy
NestJS Fastify, fastify, find-my-way, fast-uri	Alta	Actualizar y repetir controles de API
brace-expansion, nanoid, postcss, sharp, source-map-js	Alta	Actualizar transitivas y validar construcción
baseline-browser-mapping	Moderada	Actualizar y repetir auditoría

Pruebas funcionales y seguridad de API

Casos T01 a T19 con resultados reales

Ejecución en Docker local. Todas las filas siguientes tuvieron el resultado esperado. Un HTTP 400, 401, 403 o 409 constituye una prueba aprobada cuando el caso exige rechazar la operación. El archivo pruebas-api.json conserva detalle y duración de cada verificación.

Caso	Verificación	HTTP
T01	API y dependencias disponibles	200
T02	Ruta interna sin sesion rechazada	401
T03	Login invalido rechazado	401
T04	Login interno y cookies de sesion	201
T05	Creacion de tienda de demostracion	201
T06	Login del operador con permisos limitados	201
T07	Acceso a auditoria sin permiso rechazado	403
T08	Seleccion de tienda sin CSRF rechazada	403
T09	Tienda no asignada rechazada	403
T10	Seleccion de tienda asignada	201
T11	Registro de cliente y codigo generado	201
T12	Campos no permitidos en factura rechazados	400
T13	Monto negativo rechazado	400
T14	Calculo de puntos en servidor	201
T15	Factura genera movimiento de puntos	201
T16	Factura duplicada rechazada	409
T17	Cliente consulta puntos propios	200
T18	Identidad del cliente procede de su sesion	200
T19	Sesion de cliente rechazada en API interna	401

Pruebas de canjes y sesiones

Casos T20 a T37 y pruebas automatizadas

El flujo verificó reserva, aprobación, preparación, entrega, cancelación y reversa. Los casos finales probaron renovación, detección de reutilización, revocación y límite de solicitudes.

Resultado global: 37 verificaciones de API aprobadas.

Caso	Verificación	HTTP
T20	Creacion de premio con permiso	201
T21	Canje sin puntos suficientes rechazado	400
T22	Solicitud reserva puntos y stock	201
T23	Operador sin permiso no aprueba canje	403
T24	Aprobacion administrativa del canje	201
T25	Envio del premio a tienda	201
T26	Premio listo en tienda	201
T27	Entrega del premio con trazabilidad	201
T28	Segunda entrega rechazada	400
T29	Cancelacion devuelve puntos y reserva de stock	201
T30	Reversa autorizada conserva historial	201
T31	Saldo final consistente	200
T32	Renovacion de sesion valida	201
T33	Reutilizacion de refresh antiguo detectada	401
T34	Sesion revocada rechaza token de acceso	401
T35	Cierre de sesion autorizado	201
T36	Cookie antigua falla despues del cierre	401
T37	Limite de solicitudes de login aplicado	429

Resultados y conclusiones

La demostración confirma el ciclo de fidelización y su trazabilidad

Resultado funcional: una compra aprobada de Q500 generó 500 puntos. Un canje entregado consumió 200. La cancelación de otra solicitud restituyó 200. Una compra adicional de Q150 seguida de su reversa dejó efecto neto cero. El saldo final del cliente principal es 300 puntos y el del cliente alterno es cero.

La aplicación rechazó facturas repetidas, saldo insuficiente, campos adicionales, montos negativos, tienda no asignada, ausencia de CSRF y acciones sin permiso. La auditoría vincula la compra y el canje con sus actores y conserva la transición de estados.

Las 23 pruebas automatizadas existentes cubrieron cookies, segmentación por marca, registro, banners, pagos online, entrega y liquidación. Los dos chequeos de tipos terminaron sin errores. La restauración coincidió en las 53 tablas. El alcance no incluye pruebas de penetración, carga, concurrencia real ni comercio online completo en interfaz.

Aporte a la transformación digital: el cliente consulta sus beneficios, la tienda aplica reglas en servidor y la administración revisa registros en un repositorio común. Estos resultados demuestran capacidades operativas. No existe una medición antes y después para atribuir porcentajes de ahorro, incremento de ventas o mejora de retención.

Para medir beneficios tras un piloto, registrar tiempo por factura, tasa de duplicación, incidencias de saldo, tiempo de entrega, canjes completados y recurrencia de compra. Comparar períodos equivalentes y excluir los datos demo. El nivel mostrado y la equivalencia de Q3 no constituyen dinero entregado al cliente.

Conclusión: el sistema ejecuta el proceso seleccionado y ofrece evidencia de controles de seguridad. La siguiente fase debe tratar los avisos de dependencias, aplicar mínimo privilegio en base, completar respaldos de archivos y validar la infraestructura HTTPS. El proyecto académico puede exponer con precisión lo comprobado y los pendientes.

Resultado medido	Valor	Fuente
Verificaciones de API	37 de 37	pruebas-api.json
Pruebas automatizadas	23 de 23	pruebas-unitarias.txt
Saldo final del cliente principal	300 puntos	T31 y pantalla del cliente
Restauración de base	53 tablas coincidentes	restauracion-respaldo.txt
Avisos de dependencias	11	auditoria-dependencias.json

Matriz de cumplimiento y anexos

Relación entre la consigna y los archivos entregados

El manual complementa las instrucciones de acceso y operación con capturas reales. La presentación cubre los siete temas de exposición mediante nueve diapositivas y un reparto total de 8 minutos. El guion indica quién expone, qué muestra y cuánto tiempo dispone.

Los anexos incluyen capturas JPG, pruebas JSON y TXT, auditoría de dependencias, configuración no confidencial, inventario de fuente, referencias de código y diagramas UML Mermaid. El código ZIP excluye dependencias instaladas y secretos de operación.

Las capturas preservan las pantallas reales. Los datos Demo XRE distinguen las operaciones académicas de los registros previos. La prueba de despliegue disponible es local. El estado parcial de producción pública se explica en el apartado correspondiente y debe completarse con evidencia del servidor.

Requisito de la consigna	Ubicación y evidencia	Estado
Proceso y problema	Informe, proceso y solución	Documentado
Solución y tecnologías	Arquitectura y funcionalidades	Documentado
Código fuente organizado	codigo-fuente.zip y manifiesto	Incluido
Puesta en producción	Entorno local y procedimiento público	Local verificado y público pendiente
Pruebas y resultados	37 API, 23 unitarias y respaldo	Evidencia real incluida
Manual con capturas	Manual de usuario y carpeta evidencias	Incluido
Resultados y conclusiones	Resultados medidos y plan de piloto	Documentado
Identidad y autorización	Cookies, sesiones, permisos y rechazos	Verificado en casos ejecutados
Arquitectura y UML	Tres archivos Mermaid y figuras	Incluido
Datos y base	Restricciones, restauración y privilegios	Controles y pendientes documentados
DevOps y CI/CD	Auditoría npm, repositorio y propuesta CI	CI y remediación pendientes
Aplicación y producción	Validación, CSRF y guía HTTPS	Parcial según alcance
Exposición de 8 minutos	PPTX, PDF y guion por integrante	Preparada